

Genetic_Algorithm

1. 使用场景

用于指导和修改其他启发式方法，以在庞大且复杂的搜索空间中高效地找到近似全局最优解。核心应用领域是组合优化问题，特别是NP-hard问题。

面对这类挑战，我们不再执着于寻找那个唯一的、完美的“最优解”，而是转向寻求在可接受的时间和计算成本内，找到一个足够好的“满意解”或“近似最优解”。启发式优化算法正是为此而生。

2. 原理

简单来说就是模拟“优胜劣汰，适者生存”的进化过程，来搜索问题的最优解。

GA的核心思想是将问题的潜在解看作生物种群中的“个体”，每个个体由一条“染色体”来编码。算法从一个随机生成的初始“种群”开始，通过迭代的方式不断进化。在每一个generation中，算法会根据每个个体对环境的“适应度”（即解的质量）来进行选择，适应度高的个体有更大的概率被选中，并通过“交叉”和“变异”等遗传操作产生下一代的新个体。这个过程不断重复，种群的整体适应度会逐渐提高，最终收敛到适应度最高的个体，即问题的最优解或近似最优解。

GA维护一个解的“种群”，通过交叉操作在不同解之间交换信息，实现了一种**并行的**、基于群体智慧的全局搜索。它拥有对整个搜索历史的“长期记忆”。相比之下，SA在任意时刻只维护一个当前解，通过邻域操作进行局部的、**串行的**探索。它的“记忆”是短期的，仅限于当前解，但通过温度控制的概率接受机制，它获得了跳出局部最优的强大能力。

可以认为，GA更侧重于全局的“探索”；而SA更侧重于局部的“开发”，同时保留了概率性的全局跳跃能力。

3. 代码步骤

以经典的旅行商问题（TSP）举例。该问题要求一个销售员从一个城市出发，访问给定的一系列城市，且每个城市只访问一次，最后回到起点，目标是找到总路程最短的路线。虽然问题描述简单，但其求解难度随城市数量的增加呈指数级增长，但是使用GA就非常适合（SA也非常适合）。

第一步: 问题定义与数据准备

定义城市的二维坐标。

在实际建模中，这些数据可能来自问题描述或附件。

```

1  import numpy as np
2  import random
3  import matplotlib.pyplot as plt
4
5  # 城市坐标 (15个城市)
6  cities_coordinates = np.array(, , , , ,
7      , , , , ,
8      , , , , )
9  num_cities = len(cities_coordinates)

```

Fence 1

第二步: 编码与适应度函数

我们使用路径表示法（一个包含0到14的随机排列列表）作为染色体。适应度函数计算总距离的倒数。

```

1  # 计算两个城市间的欧氏距离
2  def calculate_distance(p1, p2):
3      return np.sqrt(np.sum((p1 - p2)**2))
4
5  # 计算染色体 (路径) 的总距离
6  def calculate_total_distance(chromosome, coordinates):
7      total_dist = 0
8      for i in range(len(chromosome)):
9          current_city = chromosome[i]
10         next_city = chromosome[(i + 1) % len(chromosome)] # 回到起点
11         total_dist += calculate_distance(coordinates[current_city],
12             coordinates[next_city])
13     return total_dist
14
15 # 适应度函数
16 def fitness_function(chromosome, coordinates):
17     return 1.0 / calculate_total_distance(chromosome, coordinates)

```

Fence 2

第三步: 遗传算子实现

这里我们实现锦标赛选择、顺序交叉 (OX) 和反转变异。

```

1  # 锦标赛选择
2  def tournament_selection(population, fitnesses, k=3):
3      selected_indices = np.random.choice(len(population), k, replace=False)
4      best_index = -1
5      max_fitness = -1
6      for index in selected_indices:
7          if fitnesses[index] > max_fitness:
8              max_fitness = fitnesses[index]
9              best_index = index
10     return population[best_index]
11
12 # 顺序交叉 (Ordered Crossover - OX)
13 def ordered_crossover(parent1, parent2):
14     size = len(parent1)
15     child = [-1] * size
16
17     start, end = sorted(random.sample(range(size), 2))
18

```

```

19     # 复制父代1的片段到子代
20     child[start:end+1] = parent1[start:end+1]
21
22     # 填充剩余基因
23     p2_genes = [gene for gene in parent2 if gene not in child]
24
25     child_idx = 0
26     for i in range(size):
27         if child[i] == -1:
28             child[i] = p2_genes[child_idx]
29             child_idx += 1
30
31     return child
32
33     # 反转变异 (Inversion Mutation)
34     def inversion_mutation(chromosome, mutation_rate):
35         if random.random() < mutation_rate:
36             start, end = sorted(random.sample(range(len(chromosome)), 2))
37             segment = chromosome[start:end+1]
38             segment.reverse()
39             chromosome[start:end+1] = segment
40     return chromosome

```

Fence 3

第四步: 编写主循环与求解

这是遗传算法的核心部分，我们将所有组件整合在一起。我们还将加入“精英保留”策略，即每一代都将当前最好的个体直接复制到下一代，以确保最优解不会在进化过程中丢失。

精英保留的比例一般在10%-20%。

```

1     # GA 主函数
2     def genetic_algorithm_tsp(coordinates, pop_size=100, elite_size=10,
3         mutation_rate=0.05, generations=500):
4         num_cities = len(coordinates)
5
6         # 1. 初始化种群
7         population =
8         for _ in range(pop_size):
9             chromosome = list(np.random.permutation(num_cities))
10            population.append(chromosome)
11
12            best_distance_history =
13            avg_distance_history =
14
15            print("开始进化...")
16
17            for gen in range(generations):
18                # 2. 计算适应度
19                fitnesses = [fitness_function(chromo, coordinates) for chromo in
20                    population]
21
22                # 记录当前代的统计信息
23                distances = [1.0 / f for f in fitnesses]
24                best_distance = min(distances)

```

```

23     avg_distance = np.mean(distances)
24     best_distance_history.append(best_distance)
25     avg_distance_history.append(avg_distance)
26
27     if (gen + 1) % 50 == 0:
28         print(f"第 {gen+1} 代: 最短距离 = {best_distance:.2f}, 平均距离 = {avg_distance:.2f}")
29
30     # 3. 生成新一代
31     new_population =
32
33     # 精英保留
34     elite_indices = np.argsort(fitnesses)[-elite_size:]
35     for index in elite_indices:
36         new_population.append(population[index])
37
38     # 填充剩余种群
39     for _ in range(pop_size - elite_size):
40         # 选择
41         parent1 = tournament_selection(population, fitnesses)
42         parent2 = tournament_selection(population, fitnesses)
43         # 交叉
44         child = ordered_crossover(parent1, parent2)
45         # 变异
46         child = inversion_mutation(child, mutation_rate)
47         new_population.append(child)
48
49     population = new_population
50
51     # 找到最终的最优解
52     final_fitnesses = [fitness_function(chromo, coordinates) for chromo in population]
53     best_index = np.argmax(final_fitnesses)
54     best_chromosome = population[best_index]
55     best_distance = calculate_total_distance(best_chromosome, coordinates)
56
57     return best_chromosome, best_distance, best_distance_history, avg_distance_history
58
59 # 运行GA
60 best_route, best_dist, best_hist, avg_hist = genetic_algorithm_tsp(cities_coordinates)
61
62 print("\n进化结束.")
63 print(f"最优路径: {best_route}")
64 print(f"最短距离: {best_dist:.2f}")

```

Fence 4

第五步: 结果分析与可视化

对单次运行的结果进行可视化。

```

1 # 绘制收敛曲线
2 plt.figure(figsize=(12, 6))
3 plt.plot(best_hist, label="当代引最优距离 (Best Distance)")
4 plt.plot(avg_hist, label="当代引平均距离 (Average Distance)")

```

```

5 plt.title("GA for TSP Convergence Curve")
6 plt.xlabel("Generation")
7 plt.ylabel("Distance")
8 plt.legend()
9 plt.grid(True)
10 plt.show()
11
12 # 绘制最优路径图
13 plt.figure(figsize=(8, 8))
14 ordered_coords = cities_coordinates[best_route]
15 ordered_coords = np.vstack([ordered_coords, ordered_coords]) # 添加回到起点的路径
16
17 plt.plot(ordered_coords[:, 0], ordered_coords[:, 1], 'o-', label='Optimal Route')
18 for i, (x, y) in enumerate(cities_coordinates):
19     plt.text(x, y, str(i), color="red", fontsize=12)
20 plt.title(f"Optimal TSP Route Found by GA (Distance: {best_dist:.2f})")
21 plt.xlabel("X Coordinate")
22 plt.ylabel("Y Coordinate")
23 plt.legend()
24 plt.grid(True)
25 plt.show()

```

Fence 5

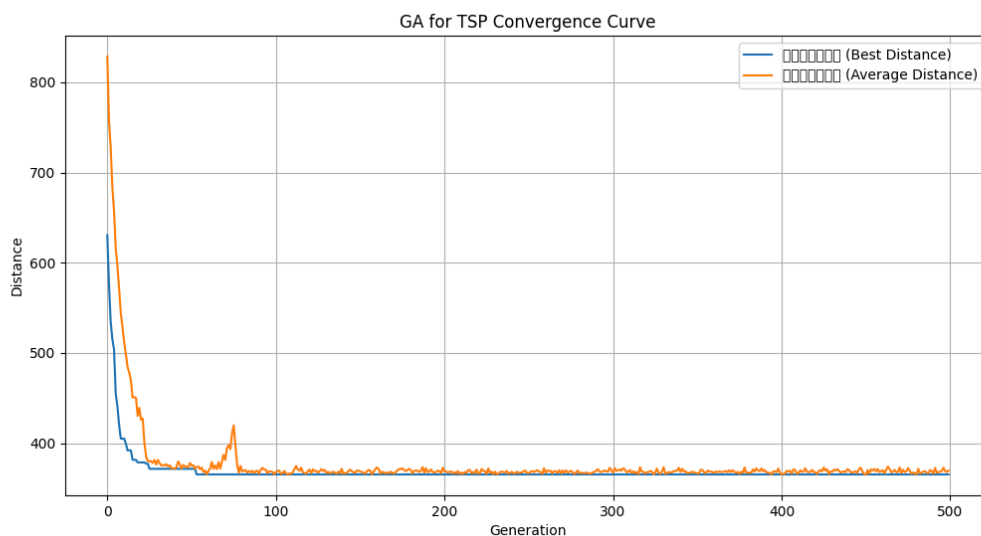


Image 1

上图是程序输出的收敛曲线图，它直观地展示了算法的进化过程：最优距离不断下降并趋于稳定，平均距离也随之下降，表明整个种群在向更优的方向进化。路径图则清晰地展示了最终找到的解决方案。

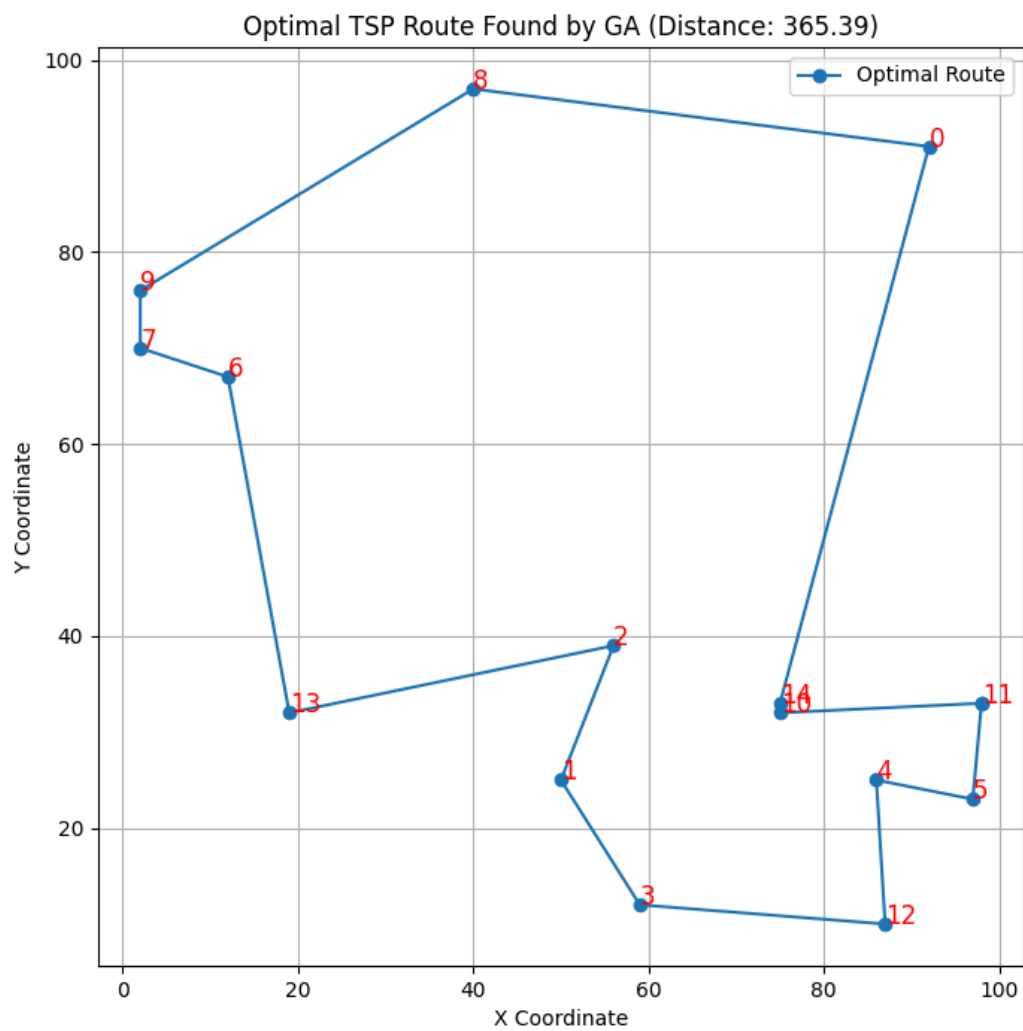


Image 2

上图是程序输出的最优路径图。